Lecture 7: General purpose input and output

Lui Holder-Pearson

September 10, 2026

Learning Goals

- Know what GPIO connections are, and why they're needed.
- Understand the special `DDRx`, `PORTx`, and `PINx` registers for GPIO ports.
- Briefly understand some electrical limitations associated with GPIO pins.
- Understand some practical implementations of interfacing the GPIO pins.

Further Reading

Rizzoni and Kearns. *Principals and Applications of Electrical Engineering*, Chapter 14.5

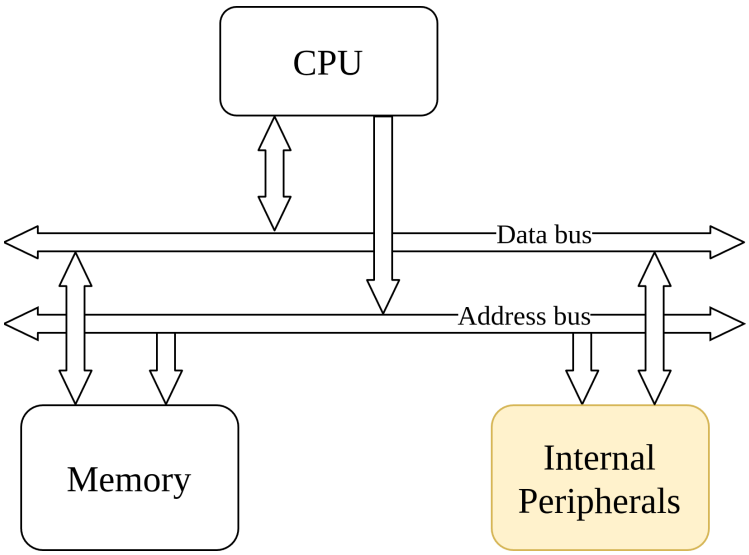
References

Ciaran Moore's past ENCE260 lecture notes
Rizzoni and Kearns. *Principals and Applications of Electrical Engineering*, Chapters 13 & 14.

Real word interaction

Embedded systems are fundamentally small computers which are capable of running independently. However, there is not much use in a small computer if it is incapable of some level of interaction.

The term *inputs and outputs* is used to describe the low-level electronic hardware to enable this interaction with the real world.



There are various different types of inputs and outputs. Some are able to be interfaced to analogue signals, some are connected to special registers for enabling communication protocols, and some have very high current or speed outputs.

Discussed further in Lecture 10: Peripherals

Most pins will be able to perform multiple functions, but the basic digital functionality is referred to as *general purpose input and output functionality*: GPIO. There are four fundamental functions of GPIO: driving `HIGH` or `LOW`, and/or reading the digital state of the pin.

1 GPIO

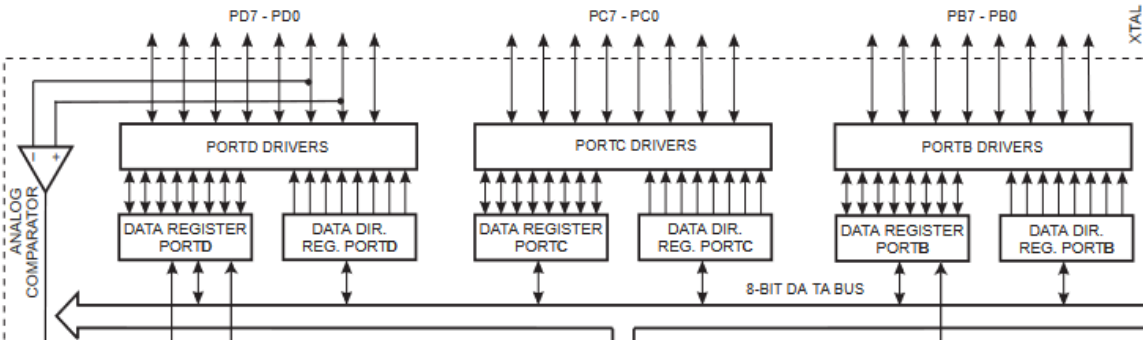
- Every microcontroller has GPIO pins.
- No microcontroller has enough GPIO pins.
- Programmable, bidirectional GPIO pins are essential to increased flexibility.

1.1 GPIO Ports

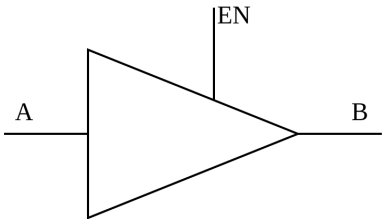
A port is a term used to refer to 8 pins. They are electrically related, but not necessarily physically close in terms of routing.

In the case of 8-bit microcontrollers, such as the ATMega32u2. Devices will usually have ports the same size as the word length.

- Generally, 8 *pins* make up a *port*.
- Pins are numbered 0-7.
- Ports are identified with letters, for example the ATMega32u2 has ports B, C, and D.
- Each GPIO port within the ATMega32u2 has three registers associated with it:
  - DDRx
  - PORTx
  - PINx

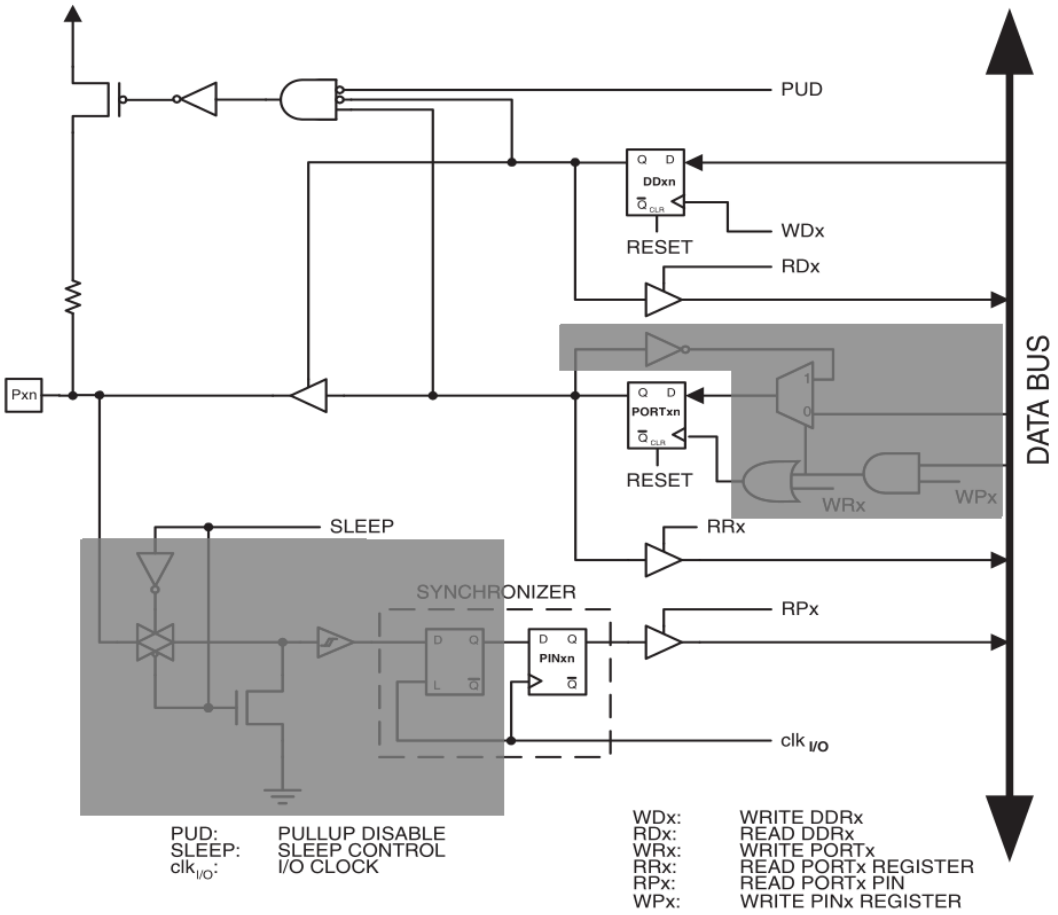


The bidirectionality of the pins is controlled using electrical signals. To understand the overall schematic, we will need to conceptually understand the tristate buffer:



EN	A	B
0	X	Hi-Z
1	0	0
1	1	1

A high impedance (Hi-Z) state is desirable because it means that it will not interfere with anything connected to `B`.



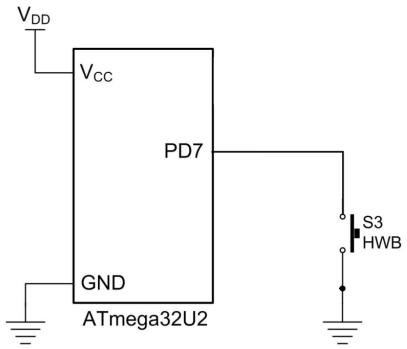
Exercise: What is the boolean expression for enabling the pullup resistor?

- `x` is the port: B, C, or D.
- `n` is the specific pin: 0-7.
- The shaded areas are of no concern.

1.2 Floating pins

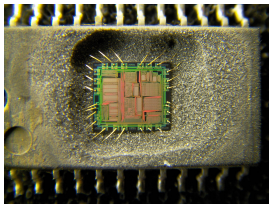
What will the value of `input` be if the button S3 is not pushed?

```
bool input;
input = PIND & 0b10000000;
```



A pin that is not definitively connected to a specific state (for digital signals this means either `0` or `1`) is said to be *floating*. Physically, these connections act as tiny antennae, and pick up electromagnetic interference from anything; adjacent pins, power supplies, or anything else.

Reminder:



To ensure that the state of the pin is known, it is connected to something. Most microcontrollers, such as the ATMega32u2, provide the ability to do this with *internal pullup resistors*, but the same functionality can be achieved with external resistors.

1.3 Current concerns

Electronics:

- Electrons — Electrically-charged subatomic particles that move around a circuit.
- Voltage — Potential energy: Energy per unit of charge.
- Current — The velocity of charge/electrons around a circuit.

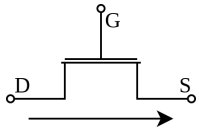
There are limits on what a microcontroller can do. The ATMega32u2:

- Can operate from a supply of 2.7 $V - 5.5$ V.
- Can provide up to 40 mA per GPIO pin.
- Can use an absolute maximum total current of 200 mA.

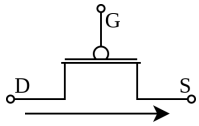
The 40 mA compares to a typical operating current (as in the current the ATMega32u2 itself consumes) of $4 - 13$ mA. Generally the maximum current is symmetrical: either in or out of the pin.

What are some options for controlling an LED from a microcontroller?

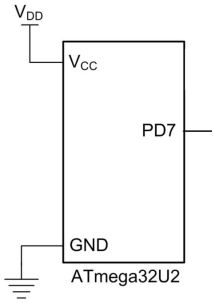
N-MOSFET, conducts when gate is `HIGH`:



P-MOSFET, conducts when gate is `LOW`



(There are many, many different symbols)



As the current requirement increases, more consideration should be given to using external driving circuitry, such as a transistor.

To manage current, it is best practice to:

1. Use some sort of drive circuitry, such as a MOSFET.
2. Use current-limiting resistors in series with the LED. The amount of added resistance will be a function of the supply voltage, the desired brightness, and the colour of the LED.

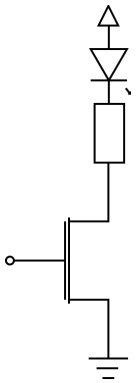
The brightness of an LED is directly proportional to the current through it. The colour of the LED matters because the forwards voltage drop is a function of the wavelength, and thus colour.

For an LED connected in series with a resistor:

$$I_{LED} = \frac{V_{DD} - V_{LED}}{R}$$

For example, a non-invasive NIR sensor is being developed with a 1650 nm LED:

Yes, they're still called '***L***ED's even if the light is not visible.



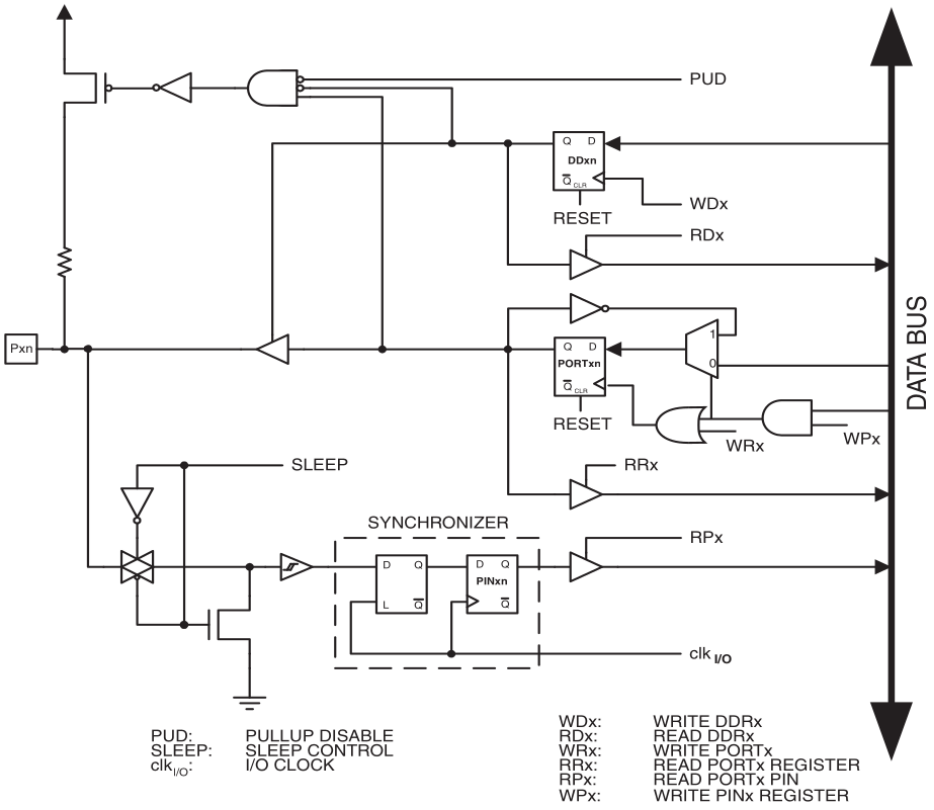
Parameter:	I_{LED}	V_{LED}	V_DD
Value:	50 mA	2.7 V	3.3 V

Exercises:

1. What is the boolean expression for enabling the pullup resistor in the ATMega32u2?

Summary

Figure 12-2. General Digital I/O(1)



Next Lecture: Lecture 8 - Sequential logic circuits

2 Questions?